

FloodEye: Real-time Flood Alert & Water Level Monitoring

IoT and Machine Learning based Early Warning System

Project Team

July 19, 2026



**Technology
Innovation Hub**
IITG TIDF



Agenda

- 1 Project Overview & Motivation
- 2 Problem Statement & Objectives
- 3 System Architecture & Workflow
- 4 Technology Stack & Hardware Components
- 5 UI Screens & Features
- 6 Machine Learning Implementation & Pipeline
- 7 Experimental Results & Limitations
- 8 Conclusion & Future Scope

- **FloodEye** is a highly responsive Progressive Web Application (PWA).
- Designed to visualise real-time water level data and environmental conditions.
- Uses an ESP32 microcontroller and multiple sensors (BMP180, DHT11, HC-SR04).
- Combines IoT telemetry, a glassmorphism UI, interactive maps, and ML.
- Aims to provide an early warning system for flood-prone areas.

- Floods cause devastating loss of life and property globally.
- Traditional monitoring systems are often expensive, slow, or inaccessible to the public.
- Need for a low-cost, open-source, and real-time monitoring solution.
- Integration of modern web technologies (Next.js) with edge IoT and browser-based AI (TensorFlow.js) bridges the gap between hardware and accessible user interfaces.

Key Value Proposition

- **Low Cost:** Utilises off-the-shelf components (ESP32, ultrasonic sensors).
- **Real-Time:** 15-second refresh interval for live telemetry.
- **Predictive:** 4-hour advanced flood warning using an in-browser 1D-CNN + BiLSTM + Attention model.
- **Accessible:** PWA allows installation on any mobile device or desktop.

Problem Statement

- Communities in flood-prone regions lack actionable, real-time data regarding local river water levels.
- Existing weather apps provide regional rainfall forecasts but lack micro-local hydrological data.
- Hardware telemetry is often disconnected from modern, user-friendly predictive analytics.
- **Challenge:** Build a reliable, low-latency, end-to-end system spanning from physical water sensors to a public-facing predictive dashboard.

Primary Objectives

- **Hardware Data Acquisition:** Continuously read water distance, temperature, humidity, and atmospheric pressure.
- **Cloud Integration:** Reliably transmit telemetry data to a cloud platform (ThingSpeak).
- **Full-Stack Dashboard:** Develop a Next.js application to visualise live and historical data.
- **Predictive AI:** Train and deploy an ML model to forecast water levels 4 hours into the future.

Secondary Objectives

- **Smart Alerts:** Implement a highly configurable alert engine with cooldowns and push notifications.
- **Offline Resilience:** Ensure the dashboard remains functional (showing cached data) even if the IoT node goes offline.
- **Data Export:** Allow researchers to download historical data (CSV/PDF).
- **Role-Based Access:** Secure administrative controls for threshold configuration.

System Architecture: Three-Tier Model

- **Hardware Layer:** ESP32 DevKit + Sensors. Uploads to ThingSpeak via HTTP REST.
- **Backend Layer:** Next.js Route Handlers fetch data, apply request coalescing, and perform caching.
- **Frontend Layer:** React/Next.js UI. Renders cards, charts, maps, and runs TF.js ML inference entirely client-side.

- ESP32 powers on and initialises sensors via I²C and digital pins.
- Main loop reads water level (ultrasonic), temperature/humidity (DHT11), and pressure/altitude (BMP180).
- Packages data into an HTTP POST payload.
- Transmits to ThingSpeak every 15 seconds.
- Features exponential backoff for Wi-Fi reconnection in case of failure.

- Client polls `/api/sensors` every 15 seconds.
- Serverless function checks in-memory cache (singleton pattern).
- If cache is fresh ($< 14s$), returns immediately.
- If stale, fetches from ThingSpeak Cloud.
- Coalescing ensures concurrent requests share a single ThingSpeak API call, preventing rate limits.

- Dashboard mounts and hydrates state from `localStorage` (offline resilience).
- `TelemetryProvider` updates global state with new readings.
- Smart Alert engine evaluates thresholds (e.g., `RAPID_WATER_RISE`).
- `useFloodPrediction` hook parses the last 48 readings.
- Features are engineered and passed to TensorFlow.js model.
- 4-hour forecast is rendered on the UI.

Hardware Components

- **Microcontroller:** ESP32 DevKit V1 (Wi-Fi enabled, dual-core).
- **Water Level Sensor:** HC-SR04 Ultrasonic Sensor (measures distance to water surface).
- **Atmospheric Sensor:** BMP180 (measures barometric pressure and altitude via I²C).
- **Environmental Sensor:** DHT11 (measures ambient temperature and relative humidity).
- **Power:** 5V USB power supply / Battery pack.

Software Technology Stack: Frontend

- **Framework:** Next.js 16 (App Router) with React 19.
- **Styling:** Tailwind CSS v4 + Framer Motion (glassmorphism & animations).
- **Charts:** Recharts for responsive, interactive SVG time-series analytics.
- **Map:** MapLibre GL JS + MapTiler for live node location overlays.
- **PWA:** `next-pwa` for service worker generation and offline caching.

- **Backend API:** Next.js Serverless Route Handlers (/api/sensors, /api/history).
- **IoT Cloud:** ThingSpeak (MathWorks) for telemetry storage.
- **Machine Learning (Training):** Python, TensorFlow/Keras, Pandas, Scikit-learn (Google Colab).
- **Machine Learning (Inference):** TensorFlow.js running in the browser (Zero server cost).

Landing Page & Scrollytelling

- An immersive, GSAP-powered root page (/).
- Features a background slideshow of flood monitoring imagery.
- ScrollTrigger and Lenis provide smooth scroll animations.
- Introduces the project, team members, and core technologies.
- Direct CTA (Call to Action) to open the Live Dashboard.

- Five real-time sensor cards showing live values, deltas, and trend arrows.
- Embedded sparkline charts for immediate historical context.
- Live prediction panel showing Risk Level (Normal, Warning, High, Critical).
- Environmental Comfort Score gauge.
- Offline banner automatically appears if the ESP32 loses connection.



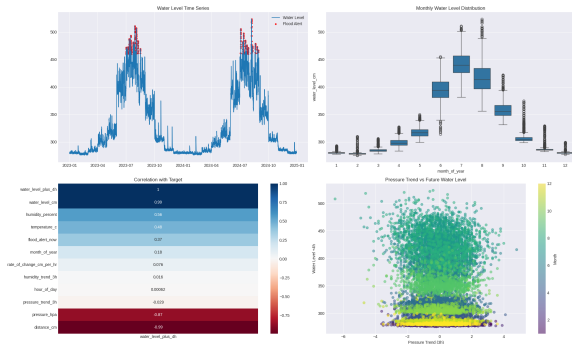
- **Analytics:** Interactive line charts for all 5 channels. Configurable time windows (1h, 6h, 24h, 7d, 30d).
- **History:** Tabular view of recent telemetry.
- Includes summary statistics (Min, Max, Avg) for the selected period.
- Features one-click export to CSV (PapaParse) or formatted PDF (jsPDF-autotable).

- **Alerts Log:** Filterable history of triggered alerts (Critical, Warning, Info).
- **Smart Push Notifications:** Alerts are sent to OS notification centre.
- **Settings (Admin):** Role-based access control. Allows tuning of danger thresholds.
- Includes a Simulation Mode to test alert generation without physical hardware manipulation.

- **Model Type:** 1D-CNN + Bidirectional LSTM + Custom Attention Mechanism.
- **Input:** 48-hour sliding window (LOOKBACK = 48).
- **Features:** 11 engineered features (4 time cyclical, 5 sensor readings, 2 trend indicators).
- **Output:** 4-hour advanced forecast of the water level (Scalar).
- **Why BiLSTM + Attention?** Captures long-term sequential dependencies and focuses on rapid changes (e.g., sudden pressure drops).

Training Pipeline (Python)

- Dataset loaded and chronologically split (70% train, 15% val, 15% test).
- Normalised using StandardScaler.
- Trained using Adam Optimizer and Mean Squared Error (MSE) loss.
- EarlyStopping prevents overfitting (patience=10); ReduceLROnPlateau adapts learning rate.
- Model evaluated on test set (MAE metric) and exported as .h5.



TFJS Conversion & Inference Pipeline

- .h5 model converted to `model.json` and `.bin` weights using `tensorflowjs_converter`.
- Deployed to `/public` directory of the Next.js app.
- **Browser Inference:** `loadModel()` fetches weights and registers the custom Attention Layer.
- Dashboard intercepts last 48 readings, engineers features on-the-fly, and runs prediction.
- Inference time is $< 100\text{ms}$ on the client device.

ML Diagnostics Panel

- Available to Admin users on the Predictions page.
- Shows active model status, tensor shapes, and live latency.
- Displays Feature Contribution breakdown (which sensor is driving the current prediction).
- Provides confidence intervals for the 4-hour forecast.

System Performance Results

- **Latency:** End-to-end data latency (Sensor → ThingSpeak → Dashboard) is approx. 1-2 seconds.
- **Load Time:** First Contentful Paint < 1.5s via Vercel Edge Network.
- **ML Accuracy:** Training Mean Absolute Error (MAE) of 3.42 cm.
- **Reliability:** Backend request coalescing successfully prevents ThingSpeak rate limits (15s minimum) even with 100+ concurrent dashboard users.

Alerting & UX Results

- Cooldown timers effectively prevent alert spam (e.g., only one RAPID_WATER_RISE alert per minute).
- Glassmorphism UI maintained stable 60 FPS during animations.
- PWA Service Worker successfully cached the application shell, allowing the app to open instantly offline.

Limitations: Hardware

- **Power Dependency:** The ESP32 requires continuous power; no solar/battery fallback implemented yet.
- **Connectivity:** Relies on Wi-Fi. Unsuitable for remote rivers without GSM/LoRa infrastructure.
- **Sensor Range:** HC-SR04 ultrasonic sensor has a limited maximum range ($\sim 400\text{cm}$) and can be affected by heavy rain or debris.

Limitations: Software & ML

- **Cold Start:** ML model requires 48 historical data points. Currently mitigated by padding with static sample data.
- **ThingSpeak Limits:** Free tier restricts updates to every 15 seconds.
- **Model Generalisation:** Trained specifically on Assam flood data; requires re-training for rivers with different topographical characteristics.

- FloodEye successfully demonstrates a low-cost, full-stack IoT solution for flood monitoring.
- The integration of edge hardware (ESP32) with a modern web frontend (Next.js) provides an accessible early warning system.
- In-browser Machine Learning (TensorFlow.js) proves to be a viable, zero-server-cost architecture for real-time predictive analytics.
- The system fulfills its primary objective of delivering actionable, micro-local hydrological data.

Future Scope: Hardware Upgrades

- Transition from Wi-Fi to **LoRaWAN** or GSM (SIM800L) for remote deployments.
- Replace HC-SR04 with industrial-grade waterproof LiDAR or radar sensors for better accuracy in heavy rain.
- Implement Solar + Li-Po battery management system for autonomous off-grid operation.

Future Scope: Software Upgrades

- Implement full PostgreSQL database (Neon) to bypass ThingSpeak's 15-second limitation and store unlimited history.
- Introduce user authentication (NextAuth) for public accounts to subscribe to specific SMS alerts (Twilio integration).
- Expand the ML model to include external weather API forecasts (e.g., OpenWeatherMap rainfall data) as additional input features.

Thank You

Questions & Answers